

BOOK OVERVIEW

따라하면서 배우는

DuckDB

게임 서버 개발자를 위한
C# 기반 데이터 분석 실전 가이드

저자: 최흥배 | AI-Assisted

C# .NET 9 · DuckDB 1.5.1 · 11장 + 부록



OLAP · 임베디드 · 컬럼 지향

게임 로그 분석

SQL 통계

C# 연동

성능 최적화

실전 프로젝트

이 책은 어떤 책인가?

"서버 없이, 파일 하나로, SQL 한 줄로, 수백만 행을 몇 초에 분석하는 데이터베이스 — DuckDB"



대상 독자

- 온라인 게임 서버 개발자
- C# / .NET 환경 사용자
- 로그 분석이 필요한 개발팀
- 별도 분석 서버 없이 데이터를
- 다루고 싶은 실무자



학습 목표

- DuckDB 개념 및 아키텍처 이해
- C#에서 DuckDB 완전 연동
- 게임 로그 설계 → 수집 → 분석
- 실무 수준의 분석 시스템 구축



개발 환경

- OS: Windows 11
- 언어: C# (.NET 9)
- DB: DuckDB 1.5.1
- IDE: VS Code + C# DevKit
- 패키지: DuckDB.NET

게임 개발자가 맞닥뜨리는 현실



방법 1: MySQL/PostgreSQL

별도 DB 서버 운영

- 집계 쿼리 시 게임 서버 느려짐
- 서버 관리 부담, 복잡한 설정



방법 2: Python + pandas

CSV를 메모리에 전부 로드

- 수GB 로그 파일은 메모리 부족
- 코드 복잡, 처리 속도 느림



방법 3: Hadoop/Spark

빅데이터 클러스터 구축

- 오버엔지니어링
- 비용과 운영 부담 폭발

공통 문제: "도구가 목적보다 크다" — 로그 몇 줄 분석하려다 인프라 구축에 매몰



▼ DuckDB가 이 모든 문제를 해결한다

DuckDB란 무엇인가?

2019년 네덜란드 CWI 연구소 탄생 · "분석(OLAP)을 위한 SQLite" · 오픈소스 임베디드 데이터베이스



심플함

설치 불필요
NuGet 패키지 하나면 충분
서버 설정 0줄



이식성

서버 없이 프로세스 내 내장
단일 .duckdb 파일
배포가 극도로 단순



성능

컬럼 지향 스토리지
벡터화 실행 엔진
수백만 행을 수초 내 처리



왜 오리(Duck)인가? 오리는 물(스트리밍), 땅(파일), 하늘(인메모리) 어디서든 적응한다.

DuckDB도 CSV · Parquet · JSON · 인메모리 데이터를 자유롭게 다루는 '만능 분석 도구'

OLTP vs OLAP — 두 세계의 구분

OLTP

Online Transaction Processing

- 실시간 게임 데이터 처리 (로그인·아이템·전투)
- 소수 행, 밀리초 단위 응답 필수
- 동시 수천 명 접근, 높은 동시성
- MySQL / PostgreSQL / Redis
- ← 게임 서버의 핵심 DB

VS

OLAP

Online Analytical Processing

- 쌓인 로그 집계·패턴 분석
- 수백만 행 스캔, 수초~수분 처리
- 읽기 위주, 처리량(Throughput)이 핵심
- DuckDB / ClickHouse / BigQuery
- ← 분석 도구의 영역

게임 서버: MySQL로 실시간 트랜잭션 + DuckDB로 분석/통계 → 두 DB는 경쟁이 아닌 역할 분담

DuckDB 핵심 아키텍처 특징

01

임베디드 (Embedded)

별도 서버 프로세스 없이 C# 앱 내에 직접 내장
NuGet 패키지 추가만으로 즉시 사용
.duckdb 파일 하나로 전체 DB 관리

```
var db = new DuckDBConnection("game.duckdb");
```

02

컬럼 지향 스토리지 (Columnar)

쿼리에 필요한 컬럼만 선택적으로 디스크에서 읽음
100개 컬럼 중 3개만 필요해도 3개만 I/O
연속된 동종 데이터 → 압축률 극대화

```
SELECT AVG(play_time), MAX(level) FROM logs;
```

03

벡터화 실행 엔진 (Vectorized)

행 1개씩이 아닌 1,024개 블록 단위 배치 처리
CPU SIMD 명령어로 대량 데이터 병렬 연산
동일 하드웨어에서 수십 배 빠른 분석 성능

```
-- 수백만 행을 수초 내 처리
```

04

인메모리 + 파일 모드 지원

":memory:" — RAM 전용, 임시 분석·테스트
파일 모드 — .duckdb 영구 저장
두 모드를 코드 한 줄 변경으로 전환 가능

```
var db = new DuckDBConnection(":memory:");
```

DuckDB vs SQLite vs PostgreSQL

SQLite

OLTP 임베디드

DuckDB

OLAP 임베디드

PostgreSQL

OLTP 서버형

항목	SQLite	DuckDB 🍬	PostgreSQL
용도	OLTP 임베디드	OLAP 분석	OLTP 서버
서버 필요 여부	불필요	불필요	필요
저장 방식	행 지향	컬럼 지향 ★	행 지향
분석 쿼리 성능	느림	매우 빠름 ★	중간
동시 쓰기	제한적	제한적	매우 우수 ★
CSV/Parquet 직접 읽기	불가	가능 ★	불가
설치 복잡도	매우 낮음	매우 낮음 ★	중간~높음
대용량 집계	느림	매우 빠름 ★	중간
게임 활용	클라이언트 캐시	로그 분석·통계 ★	계정·결제·순위

11장으로 완성되는 학습 로드맵

PHASE 1 개념 & 환경

1장: DuckDB란?

2장: 개발 환경 구축

3장: 핵심 아키텍처

PHASE 2 SQL & C# 연동

4장: SQL 기초

5장: C#에서 DuckDB 다루기

6장: 파일 읽기/쓰기

PHASE 3 게임 실전 분석

7장: 게임 로그 설계·수집

8장: 콘텐츠별 DuckDB 활용

9장: 통계 분석 실전

PHASE 4 최적화 & 완성

10장: 성능 최적화

11장: 실전 프로젝트

+ 부록 A~D

C#에서 DuckDB 다루기

연결 관리

파일 DB / 인메모리 DB
using 패턴으로 안전한 해제

DDL / DML

테이블 생성·삭제
INSERT / UPDATE / DELETE

쿼리 → 객체 매핑

C# 클래스로 자동 매핑
타입 안전한 데이터 처리

보안 & 트랜잭션

파라미터 바인딩 (SQL 인젝션 방지)
ACID 트랜잭션 처리 패턴

비동기 지원

async/await 완전 지원
게임 서버 비동기 아키텍처 적합

C# · DuckDB.NET 연동 예시

```
// 1. 연결 열기 (파일 모드)
using var db = new DuckDBConnection(
    "Data Source=game.duckdb");
db.Open();

// 2. 테이블 생성
await db.ExecuteAsync(
    """
    CREATE TABLE IF NOT EXISTS
    login_logs (
        player_id VARCHAR,
        login_time TIMESTAMP
    );
    """);

// 3. 파라미터 바인딩 INSERT
var cmd = db.CreateCommand();
cmd.CommandText =
    "INSERT INTO login_logs VALUES
    ($player_id, $login_time)";
```

온라인 게임 로그 설계와 수집



접속 로그

주요 필드
player_id ·
server_id
timestamp · ip
client_version ·
event type

분석 활용

DAU/MAU, CCU, 지역 분포



전투 로그

주요 필드
player_id ·
skill_id
target_id · damage
is_critical ·
timestamp

분석 활용

밸런스 분석, 버그 탐지



아이템 로그

주요 필드
player_id · item_id
action · quantity
timestamp

분석 활용

경제 흐름, 복제 탐지



결제 로그

주요 필드
player_id ·
product_id
amount · currency
payment_method

분석 활용

ARPU, 전환율 분석



오류 로그

주요 필드
error_code ·
message
stack_trace
server_id ·
timestamp

분석 활용

서비스 안정성 모니터링

게임 콘텐츠별 DuckDB 활용

플레이어 행동 분석

- DAU/MAU · CCU 집계
- 세션 시간 분포
- 레벨별 이탈 지점 탐지

전투 밸런스 분석

- 직업별 평균 DPS 비교
- 사망 패턴·핫스팟 분석
- 스킬 사용 빈도 통계

아이템 경제 분석

- 획득·소비·거래 흐름
- 인플레이션 감지
- 희귀 아이템 유통 추적

퀘스트/콘텐츠 완료율

- 퍼널 단계별 이탈율
- 병목 지점 자동 탐지
- 콘텐츠 완료 시간 분포

결제 & 수익 분석

- ARPU · ARPPU 계산
- 결제 전환율 코호트
- 상품별 매출 추이

이상 탐지 (Anti-cheat)

- 비정상 딜량 SQL 탐지
- 초당 이벤트 과다 플레이어
- 아이템 복제 패턴 감지

통계 분석 실전 & 성능 최적화

9장: 통계 분석 실전

내장 통계 함수

COUNT · SUM · AVG · STDDEV · PERCENTILE_CONT

윈도우 함수

순위(RANK) · 누적합(SUM OVER) · 이동 평균(AVG OVER)

시계열 분석

날짜·시간 함수로 일별/주별/월별 집계

코호트 분석

특정 날짜 가입자 그룹 추적 (D+1, D+7, D+30 리텐션)

퍼널 분석

단계별 전환율 — 접속→전투→아이템→결제

대시보드 SQL

실무에서 바로 쓸 수 있는 리포트 쿼리 모음

10장: 성능 최적화

EXPLAIN ANALYZE

쿼리 실행 계획 읽기 및 병목 지점 파악

파티셔닝 활용

인덱스 대신 날짜·서버 파티션으로 스캔 범위 감소

대용량 로드 최적화

CSV/JSON Bulk Load · COPY 명령 활용

메모리 튜닝

SET memory_limit · SET threads 설정

병렬 쿼리

멀티코어 활용 스레드 수 조정

C# 앱 최적화

연결 재사용 · 배치 INSERT · 비동기 처리

실전 프로젝트: 게임 로그 분석 시스템

가상 온라인 게임 서버 7일치 로그 생성 → DuckDB 적재 → 실무 분석 리포트 출력 (C# 콘솔 앱)



생성되는 분석 리포트

- 일별 DAU 추이 (7일)
- 직업별 평균 DPS / 중앙값 비교
- 아이템 획득·소비 상위 10종
- 일별 매출 · ARPU · ARPPU

CH.11 · 실전 프로젝트
• 이상 행동 의심 플레이어 목록

DuckDB 테이블 구조

- login_events
- battle_events
- item_events
- payment_events
- loaded_files (중복 방지)

이 책을 읽고 나면 할 수 있는 것



LEVEL 기초

- DuckDB 개념과 OLAP 아키텍처 완전 이해
- C#/.NET 9에서 DuckDB 설치 및 연동
- SQL로 CSV·Parquet·JSON 파일 직접 분석
- 파라미터 바인딩, 트랜잭션 안전 처리



LEVEL 실전

- 게임 로그 5종 스키마 설계 및 C# 라이터 구현
- 날짜별 로그 로테이션 + DuckDB 자동 적재 파이프라인
- DAU/MAU, 세션, 이탈 분석 SQL 작성
- 전투 밸런스·아이템 경제·결제 분석 쿼리



LEVEL 고급

- 코호트·퍼널·시계열 윈도우 함수 통계 분석
- EXPLAIN ANALYZE로 쿼리 플랜 읽기 및 튜닝
- Anti-cheat SQL 이상 탐지 패턴 구현
- 완전한 게임 로그 분석 시스템 처음부터 구축



DuckDB를 활용할 수 있는 곳

게임 로그 배치 분석

서버 없이 로그 파일을 SQL로 직접 분석
매일 DAU·매출·이탈 지표 자동 집계

Anti-cheat 이상 탐지

비정상 딜량·아이템 복제 패턴
SQL 쿼리 하나로 탐지 가능

관리자 운영 도구 내장

C# 앱에 DuckDB 임베드
별도 분석 서버 없이 강력한 탐색 기능

ETL 파이프라인 처리

CSV → 정제 → Parquet 변환
DuckDB 하나로 완결되는 파이프라인

개발/QA 데이터 검증

인메모리 DB로 단위 테스트
저장 데이터 정합성 즉시 검증

밸런스·이벤트 효과 측정

이벤트 오픈 후 수 시간 내
효과 수치를 로그 파일로 즉시 확인

"서버 없이, 파일 하나로, SQL 한 줄로, 수백만 행을 몇 초에 분석한다"

— 게임 서버 개발자가 DuckDB를 알아야 하는 이유